print()

Instead of writing numerical data to the SD card, we can use the `print()` function to print a string of characters to our file instead. This has a similar syntax to `write()`, as follows:

```
name.print(data)
```

Like the `print()` function for the LiquidCrystal library, data can include any text strings, bracketed by double quotes (“ and ”), and values or variables that include `char`, `byte`, `int`, `long`, or `string` data types. Any numerical value sent to the `print()` function will be converted to a text string, which is why it was necessary to use the `write()` function in our example so that we can use those values for our next example code.

Example Code: SD Flicker

Now that we have some data collected from our simple data logger stored permanently on our memory card, we can use that data to generate some form of output. The example code in Listing 9-9 will read each byte individually and write that value to the LED connected to PWM pin 9. To smooth things out some, we are using a `for` loop to fade from one value to the next.

Listing 9-9. *SD Flicker Source Code*

```
#include <SD.h>

File file;

const int interval = 10;   // time in milliseconds
const int LED = 9;
const int SDcs = 4;

byte oldValue = 0;

void setup() {
  Serial.begin(9600);
  Serial.print("Initializing SD card...");
  pinMode(10, OUTPUT);

  if (!SD.begin(SDcs)) {
    Serial.println("initialization failed!");
    return;
  }
  Serial.println("initialization done.");
}

void loop() {
  Serial.println("Reading from data.txt. ");
  file = SD.open("data.txt");
  if (file) {
    while (file.available()) {
      byte value = file.read();
```